

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Manipulation (Level 2)

Total Marks: 50

Question Count: 5

Code of Conduct

I **LEO LEENAS A (192511068)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **LEO LEENAS A (192511068)**

Assessment Tasks

Analytical Problem Solving

1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.
2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.
3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.
4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.
5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Solutions — Analytical Problem Solving

Each response is structured against the rubric criteria: Problem Interpretation, Analytical Reasoning, Method Selection, Solution Quality, and Presentation of Analysis.

Question 1 — Type-1 vs Type-2 Hypervisors

Question: Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.

Problem Interpretation

Mission-critical cloud applications demand near-continuous availability, predictable performance, strong isolation, and centralized management at scale. The core technical requirement is therefore to select a hypervisor architecture that minimizes overhead and attack surface while supporting enterprise features such as live migration, clustering, and automated failover.

Analytical Reasoning

A Type-1 (bare-metal) hypervisor — e.g., VMware ESXi, Microsoft Hyper-V, or KVM — runs directly on physical hardware, so I/O and CPU scheduling pass through a single thin control layer. This yields near-native performance because there is no intermediary host operating system consuming CPU cycles, memory, or introducing scheduling latency. A Type-2 (hosted) hypervisor — e.g., VMware Workstation, VirtualBox — runs as an application on top of a general-purpose host OS, so every virtual machine instruction is mediated twice: once by the hypervisor and once by the host OS kernel, adding I/O and context-switch overhead. From a security standpoint, Type-1 exposes only the hypervisor kernel itself, giving a comparatively small, hardened attack surface. Type-2 inherits every vulnerability of the underlying host OS (drivers, background services, user applications) in addition to the hypervisor, effectively doubling the exploitable surface — a critical weakness for regulated, mission-critical workloads. For manageability, Type-1 platforms integrate with enterprise tooling (vCenter, System Center, oVirt) enabling centralized policy control, live migration (vMotion), high availability, and resource pooling across a cluster of hosts — capabilities essential to a cloud datacenter.

Method Selection & Solution

Applying a structured comparison across performance, security, and manageability (summarized in Fig. 1) shows Type-1 outperforming Type-2 on every dimension relevant to a production cloud environment. The recommended approach is therefore to deploy a Type-1 bare-metal hypervisor across the cluster, hardened with secure-boot and TPM attestation, and centrally managed through an enterprise hypervisor manager for automated failover and load balancing.

Justification / Recommendation

Type-1 is the better choice for a cloud datacenter hosting mission-critical applications: it delivers near-native performance for latency-sensitive workloads, minimizes the attack surface by removing the host-OS layer, and provides the enterprise-grade manageability (clustering, live migration, HA/DRS) that mission-critical SLAs require. Type-2 remains suitable only for development, testing, or desktop virtualization scenarios where convenience outweighs performance and security needs.

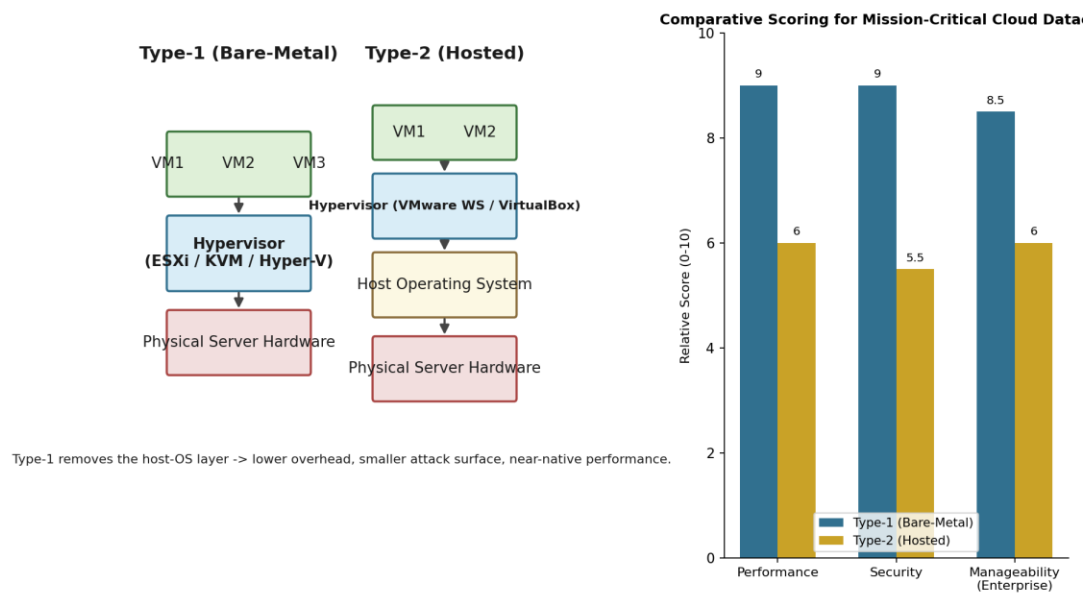


Fig. 1 — Type-1 vs Type-2 hypervisor architecture and comparative scoring for a mission-critical cloud datacenter

Question 2 — Multi-Tenant Isolation & Attack Surfaces

Question: A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.

Problem Interpretation

Co-locating finance (PCI-DSS scope), healthcare (HIPAA scope), and student-portal workloads on shared physical hardware requires strict logical isolation so that a compromise or resource contention in one tenant's workload cannot affect the confidentiality, integrity, or availability of another tenant's data — a direct application of virtualization-based multi-tenancy.

Analytical Reasoning

Virtualization enforces isolation at several layers. First, each tenant is placed in its own virtual machine with a dedicated virtual CPU, memory space, and virtual disk, so no tenant can directly address another tenant's memory — this is enforced by hardware-assisted virtualization extensions (Intel VT-x / AMD-V) and the hypervisor's memory management unit. Second, the hypervisor's scheduler enforces CPU and memory resource limits/reservations per VM, preventing a 'noisy neighbor' from starving another tenant of resources. Third, network isolation is achieved through virtual switches with VLAN or VXLAN segmentation and per-tenant firewall/micro-segmentation rules, ensuring finance, healthcare, and student traffic cannot cross tenant boundaries even though they share the same physical NIC. Despite these controls, two major attack surfaces remain: (1) Hypervisor / VM-escape vulnerabilities — a flaw in the hypervisor's device emulation or memory management could allow an attacker inside one VM to break out and access the host or a co-resident VM, defeating isolation entirely; (2) the management plane / administrative API (e.g., hypervisor console, orchestration API) — if credentials or the API are compromised, an attacker gains control over VM placement, snapshots, and network configuration across all tenants, making it a high-value, high-impact target.

Method Selection & Solution

Recommended controls, illustrated in Fig. 2, include: micro-segmentation with per-tenant VLAN/VXLAN and stateful firewall rules; hardware-assisted virtualization with IOMMU/SR-IOV for device isolation; strict resource reservations/limits to prevent noisy-neighbor effects; role-based access control (RBAC) and multi-factor authentication on the hypervisor management plane; regular hypervisor patching and secure-boot/TPM attestation to reduce the likelihood of VM-escape; and encryption of data at rest and in transit for compliance with PCI-DSS/HIPAA.

Multi-Tenant Isolation on Shared Hardware

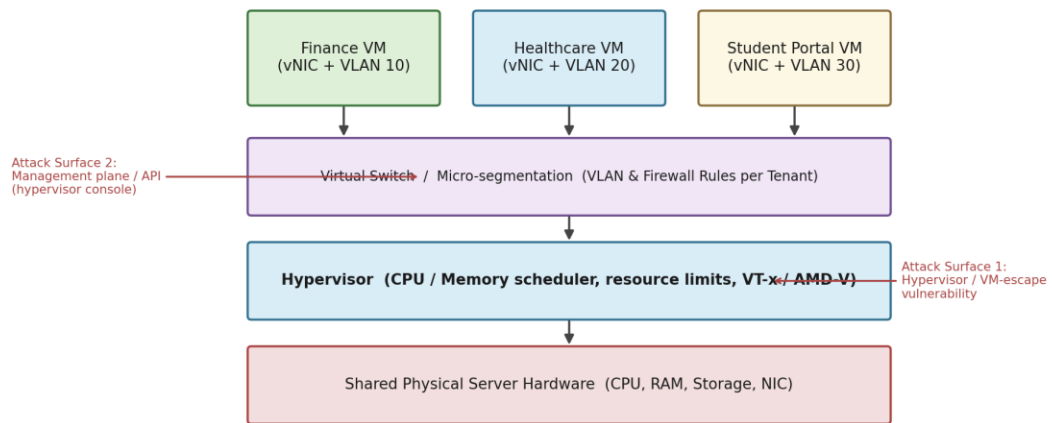


Fig. 2 — Virtualization-based multi-tenant isolation architecture with identified attack surfaces

Question 3 — VM Host Utilization Snapshot

Question: Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.

Problem Interpretation

The snapshot reports four correlated performance indicators for a single virtualization host. Each figure must be read against typical healthy-host thresholds (CPU/memory below ~80% sustained, storage latency under ~10-20 ms, VM boot time under ~60 s) to determine whether the host is resource-constrained and which subsystem is the primary bottleneck.

Analytical Reasoning

CPU at 92% is above the safe operating threshold, indicating the hypervisor's CPU scheduler is under contention — VMs will experience increased 'CPU ready time' (time waiting for a physical core), directly degrading application responsiveness. Memory at 88% is likewise near saturation; when a host approaches this level the hypervisor invokes memory-reclamation techniques such as ballooning or, worse, swapping to disk, both of which further degrade performance and can cascade into the I/O subsystem. Storage I/O latency of 35 ms is roughly 2-3x a healthy threshold for typical SSD-backed cloud storage, pointing to a storage bottleneck — likely caused by too many VMs sharing a datastore, insufficient IOPS provisioning, or the very swapping triggered by memory pressure. The average VM boot time of 170 s (versus a normal 30-60 s) is the compounding symptom: a cold boot requires simultaneous CPU, memory, and disk I/O, so when all three resources are already constrained, boot operations queue behind existing contention, producing a boot time roughly three times normal. The causal chain is therefore CPU and memory saturation → increased swapping/ballooning → storage I/O queuing → elongated boot and application response times.

Method Selection & Solution

Applying standard capacity-planning and performance-triage methodology (Fig. 3), the recommended corrective actions are: (1) use vMotion/live migration or DRS (Distributed Resource Scheduler) to rebalance VMs across additional hosts in the cluster, reducing CPU and memory pressure on the affected host; (2) right-size over-provisioned VMs and set memory reservations to reduce ballooning/swapping; (3) migrate high-I/O VMs to a faster storage tier (SSD/NVMe) or add IOPS capacity, and enable storage caching/tiering; (4) if sustained, scale out by adding a host to the cluster (horizontal scaling) rather than continuing to overcommit a single host; (5) enable proactive monitoring/alerting at 75-80% thresholds so corrective action is taken before saturation occurs.

VM Host Utilization Snapshot — Bottleneck Diagnosis

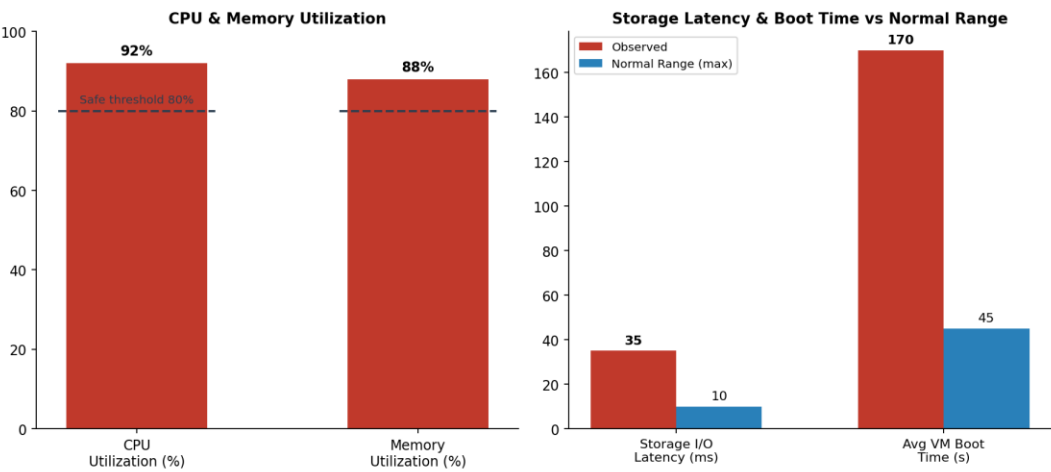


Fig. 3 — VM host utilization snapshot compared against safe operating thresholds

Question 4 — Software Defined Datacenter (SDDC) Agility

Question: *Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.*

Problem Interpretation

Agility, in a datacenter context, means the speed and ease with which infrastructure can be provisioned, reconfigured, and scaled to meet changing demand. A traditional datacenter binds applications to fixed physical hardware, so any change requires manual, hardware-level intervention; the analysis must show how software-defined abstraction of compute, storage, and network removes this dependency.

Analytical Reasoning

In a traditional datacenter, compute, storage, and network resources are siloed and manually configured: provisioning a new application typically requires racking a server, manually attaching storage LUNs, and hand-configuring switch VLANs — a process that takes days to weeks and is highly error-prone. An SDDC abstracts each of these three layers into software: Compute virtualization (hypervisors and VM/container templates) allows a new workload to be instantiated in minutes from a template rather than by physically provisioning a server. Storage virtualization (Software-Defined Storage, SDS) pools physical disks across the cluster and exposes them as policy-based, dynamically-allocated volumes, so capacity and performance tiers can be assigned programmatically instead of via manual LUN mapping. Network virtualization (Software-Defined Networking/NFV) replaces hand-configured physical switches with virtual switches, routers, and firewalls that are defined and changed through APIs, enabling network topologies to be created or modified in software without touching physical cabling. Layered above all three is an automation/orchestration layer (self-service portals, Infrastructure-as-Code, REST APIs) that ties compute, storage, and network provisioning into a single automated workflow.

Method Selection & Solution

Comparing the two architectures side-by-side (Fig. 4) across provisioning time, elasticity, and management effort shows SDDC converts infrastructure changes from a manual, hardware-bound process (days-to-weeks) into an automated, policy-driven, API-triggered process (minutes). This directly supports cloud-native requirements such as auto-scaling, self-service provisioning, and DevOps/CI-CD pipelines, because compute, storage, and network can all be requested and released programmatically as demand fluctuates.

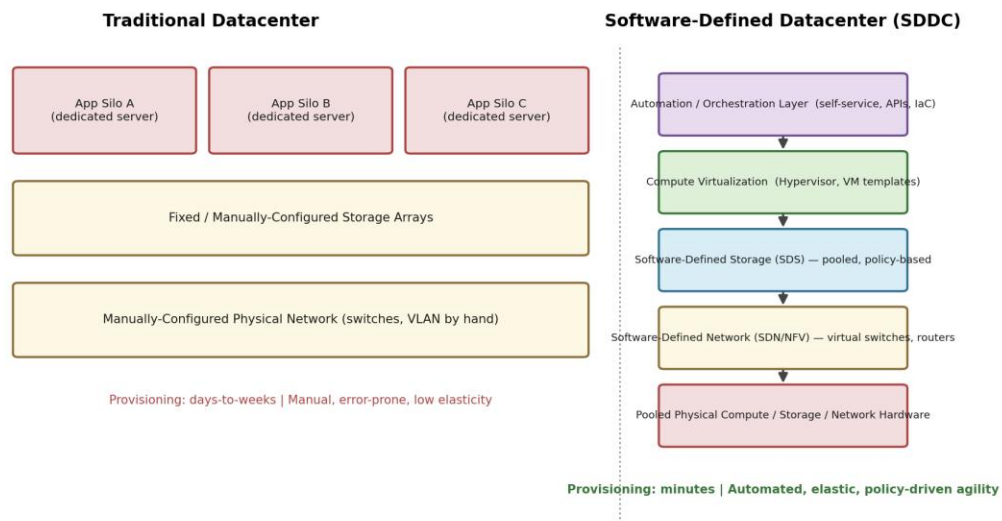


Fig. 4 — Traditional datacenter (siloe, manual) versus Software Defined Datacenter (virtualized, automated) architecture

Question 5 — Multi-Cloud Federation Identity & Privacy

Question: A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Problem Interpretation

Identity mismatch between providers means each cloud platform maintains its own Identity Provider (IdP), authentication protocol, and user-attribute schema. Without a common trust and translation layer, a user authenticated at Provider A cannot be reliably recognized or authorized at Provider B, causing federation (single sign-on across providers) to fail and creating both an availability problem and a privacy risk if attributes are exchanged inconsistently.

Analytical Reasoning

The root cause is the absence of a unified federation layer: Provider A may issue SAML 2.0 assertions while Provider B expects OpenID Connect (OIDC) tokens; even where protocols match, attribute names and formats (e.g., 'department' vs 'org_unit') may differ, so authorization decisions fail even though authentication succeeded. This also creates a privacy exposure, because without a controlled attribute-mapping layer, more user attributes than necessary may be passed to a partner provider, violating least-privilege/data-minimization principles expected under regulations such as GDPR.

Method Selection & Solution

The proposed design (Fig. 5) introduces a central Identity Broker / Federation Gateway that sits between the user and each cloud provider's IdP. The broker terminates the user's login once, then performs protocol and token translation — converting SAML assertions to OIDC tokens and vice versa — so each provider continues to use its native protocol without requiring the user to re-authenticate (true Single Sign-On). An attribute-mapping/normalization layer (supported by SCIM for provisioning) translates each provider's schema into a common attribute vocabulary, and only the minimum necessary attributes are disclosed to each relying party (attribute-based, least-privilege release) to preserve privacy. A formal trust agreement (federation metadata, mutual TLS, signed assertions) establishes cryptographic trust between the broker and each provider's IdP, and Zero-Trust principles (continuous verification, short-lived tokens) reduce the risk of a compromised token being reused across providers.

Justification / Recommendation

This identity-broker-based federation design resolves the mismatch by decoupling the user experience (one login) from each provider's native identity protocol, while the attribute-normalization and least-privilege disclosure rules directly address the privacy risk — giving the multi-cloud project a secure, standards-based (SAML/OIDC/SCIM), and auditable identity architecture.

Secure Multi-Cloud Identity Federation Design

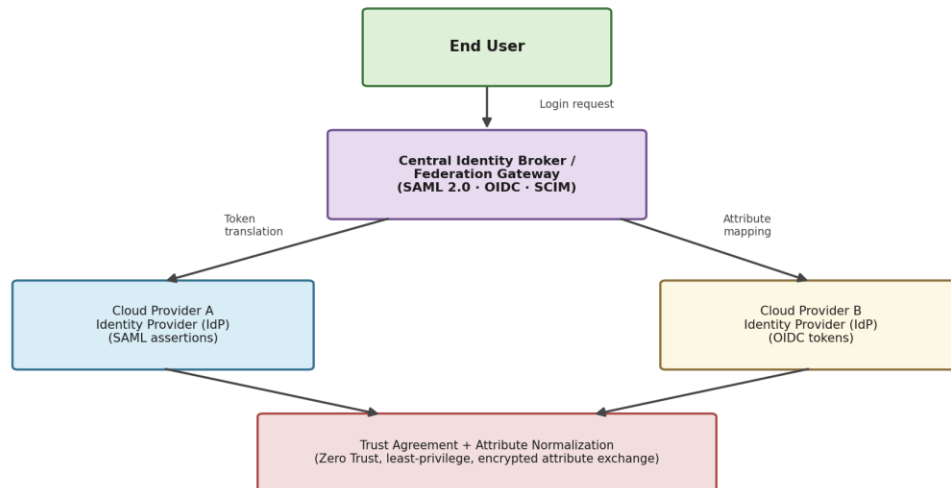


Fig. 5 — Secure multi-cloud identity federation architecture using a central identity broker

Analytical Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 – Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Interpretation	20%	Interprets all technical requirements accurately	Interprets most requirements correctly	Partial interpretation	Misinterprets the problem
Analytical Reasoning	25%	Strong reasoning with clear cause-effect analysis	Good reasoning with minor gaps	Basic analysis only	Weak or no analysis
Method Selection	20%	Chooses highly appropriate virtualization and security concepts	Chooses mostly suitable concepts	Partially suitable concepts	Inappropriate approach
Solution Quality	20%	Provides feasible and technically sound solution	Reasonable solution with minor issues	Basic solution with limitations	Unclear or invalid solution
Presentation of Analysis	15%	Well-structured and technically precise	Generally clear	Somewhat unclear	Difficult to follow

Total Marks Awarded: